Independent Reproduction and Review of LMCompress across Four Modalities

Abstract

The goal of this report is to independently implement and review the compression pipeline of LMCompress in four modalities: text, audio, image and video. In the course of the implementation I rewrote the arithmetic coder, the measurement engine and the data loaders, and designed a unified calling interface for the three models Llama, bGPT and ImageGPT. Throughout this reproduction I used the authors' released training weights and test sets so that the reproduction could be checked against them; the authors' released code was used only for reference and cross-checking.

For the text base model, Llama, I sampled from the MeDAL test set; the sampled scale is smaller than the full evaluation in the original paper. The base model reaches a rate of **9.1353%** under an 8,192-character window, a difference of **0.08 percentage points** from the 9.06% reported in the paper. The LoRA adapter released by the authors has an effect in the *opposite* direction to that claimed in the paper: it raises the rate under all five settings examined, with a penalty of **0.58–0.68 percentage points**.

For the audio model bGPT I ran a same-payload test on the LJSpeech test set. On the same 82,220-byte audio clip, the independent encoder emits 16,297.875 B against the paper's reference value of 16,399.95 B, a relative difference in compression rate of about **0.1%**.

For the image and video model ImageGPT, the image modality could not be brought into agreement with the paper, and it could not be shown that the original paper's procedure is lossless; the video modality shares the same problem of a non-lossless compression procedure. For images, the median rate over five CLIC images is **21.4781%**, roughly seven percentage points from the paper's 14.62%.

For video, the per-channel test over nine distinct frames gives a rate of **26.1866%**, also markedly higher than the paper. Neither images nor video undergo a lossless procedure: iGPT encodes through a non-injective preprocessing step that resizes to 32×32 and maps colours onto 512 palette entries, so the original pixels are not what is coded, and without side information no lossless reconstruction is possible. An exhaustive check shows that the 256 possible single-channel values map onto only **48** distinct token IDs, so the compression procedure supplied by the source code is not lossless.

This work shows that part of LMCompress can be reproduced independently: the text base rate and the audio result are close to the published values. The claimed benefit of LoRA for the text model could not be reproduced, and the image and video results are not sufficient to support the paper's conclusion that the procedure is lossless. The conclusions of the original paper need to be read modality by modality.

1 Introduction

1.1 Background

The requirement of lossless compression is unambiguous: the decompressed data must be identical to the data before compression. Lossless compression applies wherever information loss cannot be accepted—medical records and images, legal documents, scientific data, archival storage. What a lossless compressor must do is exploit as much of the predictable part of the data as possible in order to reduce the number of bits needed to represent it, while retaining all the information required to recover the original data exactly.

Shannon's information theory shows that the more predictable the data, the shorter the average code length needed to describe it. In the development of classical lossless compressors a compression system came to be split into two cooperating stages: a *predictor* that estimates the probability of the next symbol from the content already processed, and an *encoder* that uses those probabilities to assign a code length to the symbol that actually occurs. The quality of the predictor determines how closely the probabilities match the true distribution of the data, while the encoder is responsible for turning probabilities into a bitstream that can be transmitted and decoded.

Large pre-trained models give the predictor stage a new means of realisation. The statistical regularities a large model learns from massive data can be used to predict

the probability of the next symbol in a stream of text, audio or visual information. But accurate prediction does not by itself mean that the system achieves lossless compression of the source data: how the data are turned into tokens, whether the decoder can obtain the same predictive distribution, and whether the preprocessing retains the original information all determine whether exact reconstruction is ultimately possible.

1.2 Object of study and research questions

This project takes as its object of study the reproduction of LMCompress, proposed by Li *et al.* That paper combines autoregressive models suited to different modalities with arithmetic coding. Its core framework is to convert the input into a token sequence, predict the next token from the known context, and then encode the actual sequence using the predicted probabilities. The paper applies this framework to four modalities—text, audio, image and video—using a different model and data representation for each, and compares against the classical compression baselines predominant in that modality.

This reproduction of LMCompress is concerned not only with compression rate but also with whether the procedure the paper proposes constitutes a complete lossless compression system. The reproduction therefore addresses the following questions:

1. Can the main pipeline be implemented independently, without calling the authors' compression routines, and can the paper's results be reproduced under the same or similar settings?

2. What representation does the model actually encode, and does it correspond to the source data used as the denominator in the paper? Can the original input be recovered after decoding?

3. If reproduced values differ from those in the paper or in the source code, can the difference be explained by specific factors such as window length, sample, scoring position or data preprocessing?

1.3 Scope and work performed

This reproduction implements the text, audio, image and video pipelines independently and compares local results against both the formally published paper and the authors'

reference implementation. For text, the compression rate of the base model and of the model with a LoRA adapter attached are examined separately, with character windows and token windows kept explicitly distinct, and measurements completed under several conditions. For audio, the compression rate was reproduced while also checking the correspondence between the byte payload, the patches, and the output positions of the model. For image and video the per-channel input method proposed in the paper is followed, and the question of whether iGPT's palette quantisation makes the compression procedure lossless is examined further. Each modality is accompanied by a corresponding classical compression baseline, describing the behaviour of traditional methods on the same input.

Three lines of work were emphasised. First, the interface between the large-model predictor and the arithmetic coder was completed independently, so that predictions become a code length that can actually be checked. Second, the measurement basis was inspected along the whole path from data representation to coded output, paying particular attention to whether the symbols the model actually processes and the denominator used in the report refer to the same object. Third, where results differed, the respective experimental conditions were re-established so that the causes of the difference could be sorted out and explained, with results compared under identical samples and windows.

On this basis the report distinguishes three levels: the model's probabilistic prediction over the token sequence, the bitstream the encoder emits, and the ability to recover the original data from that bitstream. Only when the decoder can reproduce the prediction process, decode the tokens accurately, and the preprocessing representation retains all the information of the source data does a low rate constitute evidence of lossless compression of the source data.

2 Literature review

2.1 Scope of the review

This review follows one central question through the development of lossless compression research: how can a system use the information it already has in order to represent the next symbol of the source data in fewer bits, while retaining the ability to restore the source information exactly? Early classical compressors achieved compression by

counting repeated substrings within a window; later compressors performed prediction using hand-designed context models. In the new period in which neural networks and large language models serve as predictors, greater model scale and predictive capability, context-window settings, and application across modalities have become new questions within the same line of development.

At the same time, because lossless compression is the premise, the object obtained after decompression must be exactly the object supplied at compression. If the decoder can only restore a transformed sequence, and that transformation is many-to-one, then the original data have not been recovered. A low model loss does not by itself amount to a compressed file either: the probabilities the model supplies must be converted into a code that a decoder with the same conditions can parse correctly. Understanding LMCompress therefore requires connecting source-coding theory, prediction, practical entropy coding, representation design and experimental measurement.

This chapter follows that line: information-theoretic bounds and coding methods first, then dictionary compression and context-based compression, then learned probabilistic models and large pre-trained models, then how images and audio are converted into symbol sequences, and finally LMCompress together with the conventions needed to evaluate a reproduction.

2.2 Discrete entropy, entropy rate and coding

Shannon defined the discrete entropy of a discrete random variable through its probability mass function. This expression fixes the "lower bound" of source coding: under lossless coding the average code length cannot fall below the entropy of the source.

For sources with memory, the limit of lossless compression is determined by the entropy rate. Real data are usually not drawn independently from a fixed distribution; the probability of a symbol typically depends on context—surrounding bytes, preceding words, neighbouring pixels, or earlier samples in an audio stream. The chain rule allows the probability of a sequence to be written as a product of conditional probabilities. Put into practice for compression, this means the predictor estimates the probability of the next symbol from the symbols already obtained.

Encoders. Given the chain rule and Shannon's theory, the more accurate the predicted probabilities, the shorter the code length. The encoder's task is precisely to map the probability distribution supplied by the predictor onto a uniquely decodable code satisfying the Kraft–McMillan inequality, keeping each symbol's code length as close as possible to its information content: symbols with high probability should receive shorter codewords, and symbols with low probability may use longer ones. Following this idea, Huffman and Witten proposed Huffman coding and arithmetic coding in 1952 and 1987 respectively. The former gives an optimal prefix-code construction for a distribution over a finite alphabet, reaching a global optimum through locally optimal choices, and was used in early classical lossless compressors. Huffman coding builds an optimal prefix code for a known finite alphabet and suits symbol-by-symbol coding; it is simple and efficient, but each codeword must be an integral number of bits. When a symbol's probability approaches 1, an ordinary binary prefix code still needs at least one bit, and when many such symbols are coded separately the Huffman code length can be appreciably above the ideal information content. Arithmetic coding removes the one-bit-per-symbol floor: it does not allocate a separate codeword to each symbol but successively narrows an interval according to the probabilities of consecutive symbols. The width of the interval representing the whole sequence depends on the probability of that sequence, and a single binary number inside the interval identifies it. Engineering realisations such as range coding and binary arithmetic coding are widely used in later classical lossless compressors and in large-model-based lossless compressors.

2.3 Theoretical and practical realisation of classical compressors

The Lempel–Ziv line. Lempel–Ziv methods match repeated substrings that have previously occurred in the context; the DEFLATE specification is the typical representative. DEFLATE was proposed in 1981 and combines LZ77, proposed in 1977, with Huffman coding. LZ77 uses a sliding window to count past repeated fragments, maintaining a table that assigns a number to every (literal, length) pair; dynamically counting the frequencies of those numbers yields the probability distribution of the file. DEFLATE then hands that dynamic distribution to a Huffman tree.

Dictionary methods are particularly effective when the data contain repeated patterns: they need not estimate a complete probability distribution over all possible files. The matching rule also limits their scope, however. They readily identify exactly repeated

content but are poor at capturing semantic similarity or long-range relationships that do not manifest as repeated strings. Window size, block boundaries and dictionary-reset policy all affect how much history a compressor can exploit.

Universal modelling and coding. LZ77 compresses while it predicts, so combining LZ77 with Huffman coding does not separate predictor from encoder. In the work considered here the compression task is for the first time decomposed into two independent modules, prediction and coding. From then on the design of the predictor became a research subject in its own right.

Separating prediction from coding is very helpful for research. Replacing the predictor does not require redesigning the arithmetic coder; the same coder can be paired with a hand-built context model, a recurrent network or a Transformer. The division also gives a clearer way to compare models: first report the code length implied by the model's probabilities, then the actual coded output, and explain where the two differ.

Variable-order context models. Once predictor and encoder were decoupled, attention shifted to designing the predictor. Variable-order context models in the Markov tradition became an important line, with LOEMA, PPM and PAQ as typical representatives.

LOEMA (Local Order Estimating Markovian Analysis) was the first lossless compressor to use a variable-order Markov model as its predictor, predicting the next symbol's probability with several orders simultaneously and using the confidence of each order to decide the most suitable order locally. The confidence design relies on hand engineering, however, and predicting with several orders at once brings low speed and high computational cost.

PPM estimates the next symbol's probability using the longest-order Markov model, and when a symbol has never been seen it encodes an ESC (escape) and falls back to the next shorter order. This solves the cost of predicting with several models simultaneously, but there is no uniform answer to how much probability the escape symbol should receive; different escape designs gave different PPM variants such as PPM-A, PPM-B, PPM-C and PPM-D. PPM's fixed maximum order also prevents it from capturing longer-range dependencies.

The earliest version of PAQ resembles LOEMA in idea, retaining the predictive power of several Markov orders simultaneously, but instead of choosing the most suitable order it

mixes the predictions. PAQ's compression performance depends heavily on hand-designed implementation details.

2.4 Neural networks and modern large-model compressors

The classical methods above trace the development of the compression system and point to how the predictor should evolve: any replacement must still assign valid probabilities to every possible next symbol using the context, and produce a bitstream through reversible coding. With the rapid development of neural networks, predictors built on neural networks entered researchers' field of view.

DeepZip and Dzip. DeepZip combines a recurrent neural network with arithmetic coding and was evaluated on text, genomes and synthetic sequences. The work also observes that the average code length per symbol agrees closely with the cross-entropy, indicating that the cross-entropy loss used to train a model is essentially the same objective as the minimum code length sought in compression.

Subsequent neural compressors explored how to train or adapt the predictor for the file being compressed. DZip further combines neural modelling with adaptive and semi-adaptive training, attempting to reduce dependence on a large external training corpus. Such methods bring practical trade-offs: more adaptation to the input may improve probability estimates, but model state, training time, memory footprint and the decoder's obligation to reproduce the adaptation process all become part of the system design. A method that yields a short ideal code length may still, if compressing a medium-sized file takes days, be better suited to research than to everyday storage.

Lossless data compression with a Transformer. The paper by Izacard, Joulin and Grave was the first to extend the neural approach to Transformers, still using arithmetic coding as the encoder and evaluating on enwik8. Its results show that a purely neural predictor is competitive for lossless compression—that a Transformer can serve as an effective lossless-compression predictor. The route from lossless compression to large models was thereby fully opened.

LLMZip. Having established the effectiveness of Transformer predictors for lossless compression, research on large-model-based lossless compression concentrated on whether

pre-trained models can serve as transferable compression priors, and LLMZip is an important node on that path. In the text domain it combines LLaMA-7B with arithmetic coding and achieves high compression rates across several text datasets. Its main contribution is to show that a pre-trained model can predict probabilities effectively.

LMCompress. LMCompress, proposed by Li *et al.*, extends the large-model-predictor-plus-arithmetic- coding approach from text to images, video and audio. The report applies three models to these four modalities, with a workflow that stays consistent throughout: convert the raw data into a token sequence suitable for the model, have the autoregressive model estimate the conditional probability of the next token, and perform entropy coding with arithmetic coding. According to the reported results, LMCompress achieves better compression rates than the classical baselines on the datasets chosen for all four modalities, and breaks earlier records for lossless compression with large models.

The contribution of this work lies in its breadth: it tests whether one general idea extends from natural language to other data types. Because several modalities are covered, the raw image and audio data must first be converted into tokens the model can process, and how that conversion is done necessarily affects experimental results and conclusions. Likewise, the text tokenisation scheme, the audio patch layout, the image processor and the video frame sampling rule jointly define what the model actually evaluates.

2.5 Datasets

LMCompress's multi-modal evaluation covers images, video, audio and text.

For images it uses ILSVRC and CLIC2019 Professional, concentrating on the processing of natural images. For video it uses Xiph.org Video Test Media (Derf's Collection), which provides a variety of uncompressed YUV4MPEG test sequences spanning different resolutions, frame rates and scene content. These visual datasets supply uncompressed sequences, avoiding the influence of prior compression.

For audio LMCompress uses LibriSpeech and LJSpeech. The former consists of multi-speaker English read speech from LibriVox public-domain audiobooks; the latter is read speech from a single female speaker. Combining multi-speaker and single-speaker data lets LMCompress test compression behaviour under different speech regularities.

For text LMCompress uses MeDAL and Pile of Law. MeDAL contains large numbers

of medical abbreviations and specialised expressions, while Pile of Law collects legal and administrative text. By testing on medical and legal text—domains where lossless compression and transmission are required—LMCompress demonstrates good compression capability in practical application domains.

2.6 Summary

The literature above shows an increasingly close connection between artificial intelligence and information compression. Early work grew out of Shannon's theory and encoders, and in exploring the compression of real text it gradually produced the decoupling of predictor and encoder with research focused on the predictor; the relationship between prediction and textual context gave rise to several predictors designed in the Markov tradition. After the rise of neural networks, research using neural networks and Transformers as predictors extended into the new field of large-model-based lossless compression.

LMCompress extends this direction to several modalities, compressing text, images, audio and video with large models. Its results suggest that the semantic understanding learned by large models can fundamentally improve compression efficiency and surpass traditional hand-designed methods.

Reproducing LMCompress therefore has both technical significance and considerable academic value.

3 Method

3.1 Clean-room implementation

The implementation of LMCompress's compression logic was carried out independently. The authors' code was used in exactly four ways: (1) as a reference object for reading the logic; (2) as the source of pre-trained weights; (3) as a supplementary source of data reference; and (4) as an unmodified executable, in order to study its runtime behaviour.

3.2 Definition of the measurement basis

The important quantities used in this report are defined as follows.

1. N: the number of symbols counted in the payload. For images and video this follows the number of processed samples as defined in the paper.

2. q_t: the token probability distribution the model gives at position t.

3. ω_t: the token that actually occurs at position t.

4. C: the sum, in bits, obtained by converting the probability of each actual token under the model distribution into a code length:

$$C \;=\; \sum_{t=1}^{N} -\log_2 q_t(\omega_t). \tag{1}$$

The compression rate follows the original paper, computed as compressed bits divided by the bits of the original payload, each payload symbol counting as 8 bits when uncompressed:

$$\text{rate} \;=\; \frac{C}{8N}. \tag{2}$$

The compression ratio is the reciprocal of the rate:

$$\text{ratio} \;=\; \frac{1}{\text{rate}} \;=\; \frac{8N}{C}. \tag{3}$$

It should be noted that C is the model-implied code length computed from the model's float32 probabilities, not the final coded length of the arithmetic coder. In implementation the arithmetic coder quantises probabilities into integer CDFs, so the final output differs slightly.

3.3 Pipeline design

Following the paper and the structure of the source code, data in all four modalities pass through three stages:

1. **Tokenisation.** Raw data are converted into token sequences the model can process. Text goes through Llama's BPE tokeniser; audio fills bytes into fixed-size patches; images and video are first split by colour channel and then quantised and tokenised by the ImageGPT processor.

2. **Prediction.** Given the token sequence, the autoregressive model for the modality produces, at each position t, the distribution q_t over the next token.

3. **Coding.** An engineered arithmetic coder encodes the token sequence according to q_t.

Table 1: Models and the working points reported in the paper.

Modality	Model	Tokenisation	Reported setting
Text	Llama-3-8B ± LoRA	BPE (Llama-3 vocabulary)	character windows 4,096/6,144/8,192; toke
Audio	bGPT (byte level)	byte → 16-byte patch	mono 8-bit PCM; prediction per patch
Image	iGPT-small	32×32, 512-colour palette	single-channel patch replicated to greyscal
Video	iGPT-small (per frame)	as image, per frame	per frame, per channel; no inter-frame info

In what follows, LMCompress* denotes the text model without fine-tuning, and LMCompress the text model with the LoRA adapter attached.

3.4 Integer range coder

Our arithmetic coder uses fixed-point integer arithmetic to subdivide the unit interval. The coder maps the whole probability range onto the integer range $[0, 2^{\text{precision}})$ and then codes using the interval corresponding to the symbol. For an interval with bounds lo and hi, writing the current width as width $=$ hi $-$ lo, the new interval is

$$\text{lo}' = \text{lo} + \left\lfloor \frac{\text{width} \cdot \text{cdf}[s]}{F} \right\rfloor, \tag{4}$$

$$\text{hi}' = \text{lo} + \left\lfloor \frac{\text{width} \cdot \text{cdf}[s+1]}{F} \right\rfloor, \tag{5}$$

where F is the full integer range and cdf the integer cumulative distribution over the alphabet.

The implementation was checked for basic correctness: every symbol is assigned a non-zero integer interval, guaranteeing that a very small probability is not rounded to zero under finite precision. The interval update is checked before output, and an exact round-trip test was performed to confirm that the coding interval remains valid after every update and that the symbol sequence recovered matches the input exactly.

3.5 Text tokenisation and windows

Text is tokenised with the BPE tokeniser of Llama-3-8B. The paired evaluation uses three character windows (4,096, 6,144, 8,192) and two token windows (4,096, 8,192); the two window units are reported separately. Each target token at position t is paired with the causal distribution at position $t - 1$, and the independent code checks that targets and token positions are aligned.

3.6 Audio byte patches

Following the original design, audio is processed as a byte sequence. bGPT uses 19 bytes per patch and returns 17 slot predictions per patch, including the predicted first byte of the next patch. The reproduction computes the probability the model assigns to each actual byte using the corresponding patch and slot index. Where a patch cannot be filled, the reference implementation pads the file to a multiple of 16 bytes so that every patch is full, whereas our implementation fills the final incomplete patch with the special value 256. Although the padding differs, it has no effect on the resulting compression rate.

3.7 Image and video

Images are predicted with iGPT-small. As described in the paper, each step processes a single colour channel of the original image, first cutting that channel into 32×32 patches. To make the single-channel data conform to the RGB three-channel format iGPT expects, the code replicates each pixel value three times, i.e. a grey pixel whose R, G and B values are equal. This yields 1,024 quantised tokens. Video reuses this per-channel path frame by frame.

The official image path computes the code length of the token at position t from the output distribution at position t, and because that distribution already contains information about the current token it does not satisfy the causal order required for step-by-step decoding. In the independent implementation I changed this to predict the token at position t from the output distribution at position $t - 1$.

3.8 Classical compressor baselines

I also reproduced the classical compressors used for comparison in the paper, including OptimFROG, WavPack, FLAC and FFmpeg, tested on the datasets supplied with the paper. The results show that these compressors handle the files correctly and that the computed rates are within a reasonable range and similar to the paper's, indicating that the rate computation is sound.

3.9 Paper results used as the reference target

The original paper's results, and where they appear, are given here as the reference standard. "CR" in the paper denotes the compression rate (compressed/original, lower is better); the reciprocal (ratio) is also tabulated.

It should be added that the preprint and the formally published *Nature Machine Intelligence* version differ in MeDAL sample size and in the fine-tuning results, so all results here follow the formally published supplementary information (SI).

Table 2: Results taken from the paper as the reference standard.

Source	Item	Rate (%)	Ratio (×)
SI Table 6	MeDAL, LMCompress* (base)	9.08	11.01
SI Table 6	MeDAL, LMCompress (LoRA)	9.06	11.04
SI Table 6	Pile of Law, LMCompress*	5.86	17.06
SI Table 6	Pile of Law, LMCompress (LoRA)	4.90	20.41
SI Table 6	MeDAL, zpaq / brotli / 7z / zstd	25.11 / 27.93 / 30.25 / 30.45	—
SI Table 3	CLIC2019, LMCompress-igpt	14.62	6.84
SI Table 3	CLIC2019, PNG / WebP / JPEG-XL	47.08 / 35.56 / 31.16	—
SI Table 4	Video high-resolution, LMCompress-igpt	15.31	6.53
SI Table 4	Video high-resolution, H.265 / H.264 / FFV1 / zstd	21.69 / 37.42 / 40.16 / 51.04	—
SI Table 9	MeDAL at context 8,192 / 6,144 / 4,096	9.06 / 9.16 / 9.36	—

3.10 How the reproduction was verified

The principal values listed in this report can be re-checked against the raw measurements in the appendix and the scripts listed there. Self-checks specific to the independent implementation were also performed: whether the encoder decodes back to the source data, whether the text prediction positions are aligned, whether the mapping from palette pixels to tokens behaves as expected, and whether the independent results agree with the reference values.

Table 3: What was measured, and on what data.

Modality	Data	Scale	Note
Text	MeDAL sample (3 files)	312,053 B payload, 57,547 tokens	paired window evaluation; the paper uses a larger corpus
Audio	LJ001-0004.wav	82,220 B, 16 kHz mono 8-bit PCM	single clip; the paper's result comes from its LJSpeech evaluation
Image	CLIC professional validation (5 images)	40,648,704 counted source samples	the model processes per-channel patches; corpus scale differs from the paper
Video	Netflix Aerial 4K clip	9 distinct frames; 26,542,080 samples per frame	per-channel model evaluation; classical baselines cover 10 frames of the same clip

4 Results and analysis

4.1 Overview

4.2 Text

Three MeDAL files, 312,053 B of payload in total. The paired evaluation covers the 4,096/6,144/8,192 character windows and the 4,096/8,192 token windows.

The results for the five paired window settings are as follows.

On the three MeDAL files the compression rate of the base model LMCompress* varies with the window setting over 8.8038 %–9.4240 %. The result under the 8,192-character window, which corresponds to Supplementary Table 9 of the paper, is 9.1353 %, about 0.05 percentage points from the paper's 9.08 %.

4.2.1 Evaluation of the LoRA adapter

In the data source used here—SI Table 6 of the formal publication—fine-tuning on MeDAL gives LMCompress* a rate of 9.08 % and LMCompress with the LoRA adapter 9.06 %, a gain of 0.02 pp or 0.2 % relative. On Pile of Law the reported result improves from 5.86 % for the base model to 4.90 %, a relative gain of 16 %.

The training parameters given in the paper are: 64 MB training / 16 MB validation / the rest test; LoRA rank 8, $\alpha = 32$, batch size 1 with gradients accumulated every 8 steps; 10 epochs; learning rate 10^{-4}; the first 10 % as warm-up; the base model loaded in 8-bit and trained in bfloat16.

Table 4: Summary of results by modality.

Modality	Object	Current result	Reproduction verdict	Paper / note
Text	base Llama-3-8B	A100 paired: 8.8038 %–9.4240 %; 9.1353 % at 8,192 characters; independent port 9.0410 %	close to the paper	SI Table 9: 9.06 % (+0.08 pp); SI Table 6: 9.08 % (−0.04 pp)
Text	authors' released LoRA	rate rises under all five settings; +0.58–0.68 pp	direction differs from the paper	SI Table 6: 9.08 % → 9.06 % (−0.02 pp)
Text baseline	brotli / zstd / 7z / zlib	fixed-window results in the baseline table of this chapter; brotli q11 gives 31.84 % at 8,192 B	close to the paper's window grid	windows 8,192/16,384/32,768 B; unbounded listed separately
Audio	bGPT	port 19.82 %; reference implementation 19.94 %	same payload, close	paper LJSpeech: 18.26 %
Image	ImageGPT causal path	CLIC five-image weighted 24.9892 %; median 21.4781 %; 20.7429 % excluding the highest	source image not recoverable	palette quantisation is many-to-one
Image baseline	WebP / JPEG-2000 / PNG	39.36 / 50.22 / 76.85 %	classical codec results	12 CLIC images
Video	ImageGPT, per frame, per channel	9 distinct frames, 26.1866 %	rate of the quantised representation	paper 15.31 %; locally 10.88 pp higher
Video baseline	H.265 / H.264 / FFV1 / zstd	26.47 / 34.09 / 40.11 / 46.29 %	source-video coding baseline	10 frames of the same clip

Table 5: Text rates by window unit and length.

Window unit and length	Base rate (%)	+LoRA rate (%)	Change (pp)	Effective context
characters: 4,096	6.5322	6.9319	+0.3997	about 739 tokens/window
characters: 6,144	6.3927	6.7958	+0.4031	about 1,130 tokens/window
characters: 8,192	6.3321	6.7308	+0.3987	about 1,477 tokens/window
tokens: 4,096	6.1657	6.6107	+0.4450	about 3,837 tokens/window
tokens: 8,192	6.1023	6.5751	+0.4728	about 6,394 tokens/window

Table 6: Text results across the five paired window settings.

Statistic	Local result	Paper / reference result	Note
Base model, five window settings	8.8038 %–9.4240 %	8,192 characters: 9.1353 %; independent port: 9.0410 %	window units listed separately
+LoRA, five window settings	9.4859 %–10.0006 %	8,192 characters: 9.5154 % (reference script)	paired measurement within the same corpus
Adapter change in rate	+0.5752 to +0.6821 percentage points	+0.5901 pp (reference script)	positive under all five settings
Directional consistency	rate rises under all five settings	same direction as the reference script	local sample is three files

During the reproduction we verified this conclusion in two ways.

1. **Using our independently written program.** With the three MeDAL files, the same window sizes as the code path, and every parameter following the paper, differing only in `lora_path`, the rate rises under all five window settings, by 0.3987–0.4728 percentage points.

2. **Testing the source code.** Under the 8,192-character window we swapped `lora_path` in the authors' released program and measured the same data. The cross-check is as follows.

Table 7: Cross-check with the authors' program at 8,192 characters.

Arm	Official ratio	Rate (%)
Base, no adapter	11.204097	8.9253
+ authors' released LoRA adapter	10.509252	9.5154
Difference	−0.694845	+0.5901 (worse)

The official code exhibits the same trend as the independent code when the LoRA adapter is used, namely an effect in the opposite direction to the paper. The next subsection, on the training log, shows a possible reason.

4.2.2 Training log and checkpoint choice

The adapter's own `trainer_state.json` records the official training trajectory:

The JSON file shows that the released adapter corresponds to the step-5120 checkpoint, whose evaluation loss is about 8.2 % above the lowest value reached during training. The

Table 8: Training trajectory recorded in the released adapter's `trainer_state.json`.

Step	Epoch	Eval loss
20	0.04	2.7491
1020	2.0	2.4708 (best)
5120	10.0	2.6728 (degenerated by 8.2 %)

training log alone is not sufficient to determine the cause of the discrepancy between the paper's results and the results obtained from both codebases, but over-fitting during LoRA training is a plausible contributing cause.

4.2.3 Limitations

The tables above show that window length affects the base model's rate, but under a given window setting the compression penalty introduced by LoRA is larger than the difference between adjacent window lengths.

The text tests were carried out with bfloat16 model weights and a corpus consisting of three MeDAL files. Because the same corpus is used throughout, the results serve to compare the base model and the adapter on the same data and under the same window conditions, but they do not represent the overall compression rate of the full MeDAL test set.

4.3 Audio

The audio reproduction uses LJ001-0004.wav, 82,220 bytes in total.

Table 9: Audio results for the same 82,220-byte payload.

Implementation	Coded output	bits/byte	Rate (%)	Ratio (×)
Our port (real arithmetic-coder output)	130,383 bit = 16,298 B	1.5858	19.82	5.046
Reference implementation (unmodified)	16,399.95 B	1.5957	19.94	5.016
Paper (LJSpeech)	—	1.4608	18.26	5.477

The baseline comparison is as follows.

The audio reproduction succeeds. For the same 82,220 B payload the independent encoder emits 130,383 bit, i.e. 16,297.875 B, while the authors' code reports 16,399.95 B; the two rates differ by about 0.1 %. The authors' code result differs from the value the paper reports on LJSpeech by about 1.68 %, which is likewise attributable to a difference in sample size.

Table 10: Audio baselines.

Baseline	Rate (%)	Paper (%)	Difference (pp)
OptimFROG `--preset max`	27.311	25.42	+1.89
WavPack `-hh -x3`	35.303	33.15	+2.15
FLAC `-compression_level 12`	45.334	41.26	+4.07
zlib level 9	62.627	—	—

In agreement with the paper's conclusion, using a large model as the predictor for audio markedly outperforms classical compressors, improving the rate by about 8 % relative to the best classical compressor.

4.4 Image

The image reproduction uses five images from the CLIC professional validation set. On these images the model generates 32×32 token patches sequentially along the per-channel path supplied by the official code. In the independent implementation I use the causal distribution at the preceding position to compute each next token's code length, in bits; per-image results are in the table below.

Table 11: Per-image causal rates on the five CLIC images.

Image	Causal rate (%)
alberto-montalesi-176097	21.4781
alejandro-escamilla-6	17.5025
ales-krivec-15949	18.8136
alexander-shustov-73	41.3849
allef-vinicius-109434	24.8664
Sample-weighted mean (5 images)	24.9892
Median	21.4781
Weighted mean excluding the highest	20.7429

The denominator N determines the amount of uncompressed data the rate is taken relative to. In image and video processing the ambiguity in the definition of that denominator plays a misleading role. In the authors' public code `compress_with_igpt.py`, the program accumulates sample counts by height and width for each input object. The public preprocessing function `process_img` handles each colour channel separately: it flattens each channel and cuts it into 32×32 single-channel patches, and because iGPT requires RGB input it replicates the single-channel value into all three channels, forming a greyscale RGB input. The ImageGPT image-processing function then quantises and tokenises this

into 1,024 tokens.

Because each channel value is 8 bits, the accumulated sample count simultaneously represents the number of bytes of source data actually processed, namely $32 \times 32 \times 5733 = 5{,}870{,}592\,\mathrm{B}$. Since one original channel sample corresponds to one token, the token count also happens to be 5,870,592. But the tokens are the result of iGPT's palette quantisation and are not the original channel values. This coincidence led the original authors to believe that images and video can be reconstructed losslessly, whereas iGPT's non-injective quantisation and the results of my decoding tests show that the process is not lossless.

In addition, the official computation of the 15.5144 % rate is in error: the source code uses the output distribution at position t to encode the token at position t, and because that distribution already contains information about the current token the decoder cannot obtain it from what has already been decoded, so the requirement of causal coding is not met. That result can be reproduced with the source code's procedure, but when the procedure is corrected to predict the token at position t from position $t - 1$, the image rate becomes 23.9264 %.

4.4.1 The iGPT processor and quantisation

The ImageGPT processor reads its configuration from the checkpoint and then uses that fixed configuration for every image; it is not retrained, the palette is not adjusted, and the processing rules are not changed. The file `preprocessor_config.json` supplies the details: 512 cluster centres, with `do_color_quantize` and `do_resize` both true, so both quantisation and resizing must be enabled.

This processor is iGPT's own tokeniser, responsible for converting two-dimensional visual data into a one-dimensional pixel sequence; iGPT is an autoregressive model over discrete colour tokens and cannot operate without this component. The component quantises every input pixel into a 512-colour palette, and that palette *is* iGPT's vocabulary, so the model's predictive capability cannot be used while bypassing the quantisation.

4.4.2 Image baselines

For the same test images and the same harness, the classical baselines are as follows.

Overall the baselines are reproduced in reasonable agreement with the paper's data, and the compression rate achieved by the large model is markedly higher than the classical

Table 12: Image baselines on the same five CLIC images.

Method	Local CLIC rate (%)	Paper CLIC2019 (%)
WebP (lossless)	39.362	35.56
JPEG-2000 (lossless)	50.215	48.64
PNG (ffmpeg, compression level 100)	76.848	47.08

baselines. But because the images processed by iGPT do not undergo a lossless procedure, this comparison is fundamentally unfair.

4.4.3 The premise of per-channel processing

As noted earlier, the original paper does not feed a mixed RGB three-channel input into iGPT; it encodes each channel independently. The paper's stated reason is:

> "Because the tokens of iGPT are greyscale values, we regard an image as three greyscale images, one for each RGB channel ... During decompression, each channel is decompressed independently, and the three channels are then merged to reconstruct the original image."

I checked this premise against the checkpoint's own palette and found it to be incorrect. The 512 cluster centres of iGPT are in fact the result of quantising full RGB colours: all 512 palette entries are RGB triples, and *none* of them is a grey colour. Even under the near-grey criterion set out in the paper, only 61 are close to grey. This shows that iGPT's tokens are not greyscale values; the per-channel method does not introduce serious error, but adopting it is not necessary. The check is tabulated below.

Table 13: Inspection of the iGPT palette.

Check	Result
Shape of `clusters`	$(512, 3)$ — 512 RGB triples
Entries exactly grey ($R = G = B$)	0 / 512
Near grey (maximum channel deviation < 0.05)	61 / 512

4.4.4 Reversibility of the palette mapping

Can the original channel value be recovered uniquely from the token alone, once it has passed through the iGPT processor? This subsection demonstrates empirically that it cannot, by enumerating all possible inputs and by inspecting real image patches.

The image loader first splits the image by colour channel, so `language_model.compress` receives a tensor of shape $(32, 32, 1)$. Because the last dimension is 1, the code replicates that channel into the three RGB positions and hands a tensor of shape $(32, 32, 3)$ to the processor. Every 8-bit channel value v therefore becomes the greyscale triple (v, v, v), which is then mapped to a token ID; the encoding stores only that token ID and does not retain the original channel value.

For a greyscale 8-bit channel the input range is $(0, 255)$, i.e. 256 values, yet after palette quantisation these 256 values use only **48** of the 512 tokens. Next, in a value-by-value enumeration the program groups by token ID and counts collisions: if a token receives k input values then those inputs form $\binom{k}{2}$ colliding pairs. Over the 48 tokens there are **585** colliding pairs in total. The largest groups—tokens 391, 324, 500, 389 and 48—each contain 7 input values. The same many-to-one mapping occurs on real images.

The examples above show that iGPT's palette quantisation is not injective, so the input value cannot be recovered uniquely from the output token, and the present compression pipeline transmits no additional residual information that would let the decoder decode losslessly.

4.5 Video

4.5.1 iGPT results

Table 14: Video results.

Quantity	
Coverage of the iGPT measurement	
Processing units per frame	
Samples per frame counted in the denominator	
Per-channel causal rate	
Paper SI Table 4 high-resolution result	15.
H.265 baseline on the same clip	26.467 % (numerically 0.2804 pp higher; the objects coded di
Object coded	32×32, 512-colour palette token representation; not

The iGPT per-channel measurement covers 9 distinct frames and gives a sample-weighted rate of 26.1866 %, with a frame-to-frame range of 0.0316 percentage points. The paper's result is 15.31 %, about 10.88 percentage points lower than the independent measurement. The cause of this gap is the same as for the image modality: the official public code shares `compress_with_igpt.py` and `language_model.compress()` between

images and video, so the conclusion that using position-t information to estimate the code length of the token at position t makes the computed result optimistic applies to video as well.

Moreover, since both image and video use iGPT as the predictor, the conclusion that palette quantisation makes the compression procedure non-lossless also applies to the video modality.

4.5.2 Classical video compression baselines

The classical video baselines cover 10 frames of the Netflix Aerial 4K clip, whose source format is 4096×2160, 60 fps, YUV420. The results are as follows.

Table 15: Classical video baselines on the same clip.

Encoder	Compressed size (B)	Rate (%)	Time (s)	Paper (%)
H.265 crf 0 (x265 veryslow)	35,124,565	26.467	154.4	21.69
H.264 crf 0 (x264 veryslow)	45,236,644	34.087	5.5	37.42
FFV1 level 3	53,234,817	40.114	1.8	40.16
zstd `--ultra -22` (raw YUV)	61,434,434	46.292	119.1	51.04

The classical baselines are reproduced essentially successfully, with rate differences fluctuating between 0.06 % and 4.7 %.

A Translator's notes: discrepancies found in the source

The following are internal inconsistencies in `CN.docx`. They are recorded here rather than silently corrected, because the source was not to be modified.

1. **Table 5 ("Text rates by window unit and length") is inconsistent with Table 6 and with the running text.** Table 5 gives the base rates as 6.5322 / 6.3927 / 6.3321 / 6.1657 / 6.1023 and the penalties as +0.3997 to +0.4728, whereas Table 6 gives the base range as 8.8038–9.4240 % and the penalties as +0.5752 to +0.6821. The factor between the two sets is $\log_2 e = 1.4427$: the Table 5 values are natural-log code lengths ("nats") rather than bits. One of the two tables should be brought into line with the other.

2. **Section 4.2, item 1** states the penalty as "0.3987–0.4728 percentage points", which repeats the un-converted values, and so conflicts with Table 6 in the same section.

3. **Section 4.2** says the 8,192-character result "differs by about 0.05 percentage points" from the paper's 9.08 %, whereas the abstract and Section 4.1 give +0.08 pp against 9.06 %. Both comparisons are defensible (9.08 is the Si Table 6 base row, 9.06 the Si Table 9 context-8,192 row) but the report should say which row is being compared.

4. **Section 3.6** says bGPT uses "19 bytes per patch" while returning "17 slot predictions". A 17-slot row implies a 16-byte payload with one additional predicted byte, so 19 appears to be a typo for 17, or the patch definition needs restating.

5. **Section 4.4.4** writes the corrected image rate as "23. 9264 %" with a space inside the number.

6. **Section 4.4.3** says "about seven percentage points" from the paper's 14.62 % for a median of 21.4781 %; the difference is 6.86 pp, and the weighted mean is 10.37 pp. Stating which statistic is meant would remove the ambiguity.